Rapport d'analyse statique - MyPizzaWorld

Informations générales

- **Date d'analyse :** 11 Mars 2026
- **Analyste :** Belrhazi Ahmed
- **APK analysé :** app-debug.apk (SHA-256: f5d75d16111f18a12a2137c77e67e788a18bf0e7445afd45d8a7df2d4fbb00d1)
- **Version :** 1.0 (Package: com.example.mypizzaworld)
- **Outils utilisés :** MobSF v4.0.6 dans VM Mobexler

Résumé exécutif

L'analyse statique de l'application MyPizzaWorld révèle un niveau de risque global **élevé** (Score MobSF : 31/100, Grade C). Les principales vulnérabilités concernent des configurations de déploiement non sécurisées dans le Manifest, laissant l'application en mode débogage et en mode test, ce qui expose considérablement sa surface d'attaque. De plus, l'autorisation de sauvegarde des données (allowBackup) représente un risque de fuite d'informations. Une révision complète des configurations de build est nécessaire avant toute mise en production.

Vulnérabilités critiques

1. **Mode Débogage Activé (Debuggable)**

- **Sévérité : Élevée**
- **Catégorie MASVS :** MSTG-CODE-2 *(L'application ne doit pas contenir de fonctionnalités de débogage)*
- **Description :** Le mode débogage est explicitement activé dans la configuration de l'application, ce qui est strictement proscrit pour un environnement de production.
- `Preuve : Fichier AndroidManifest.xml, attribut android:debuggable="true"`
- **Impact :** Un attaquant ou un reverse-engineer peut facilement attacher un débogueur au processus de l'application, inspecter la mémoire, dumper la pile d'exécution (stack trace) et manipuler le flux d'exécution de l'application en temps réel.
- **Remédiation :** S'assurer que l'attribut android:debuggable est défini sur "false" ou retiré lors de la compilation de la version de production (Release build).

2. **Mode Test Activé (Test Only)**

- **Sévérité : Élevée**
- **Catégorie MASVS :** MSTG-CODE-2 *(Le code et les paramètres de test doivent être supprimés)*
- **Description :** L'application est compilée avec le drapeau de test activé.
- `Preuve : Fichier AndroidManifest.xml, attribut android:testOnly="true"`
- **Impact :** Ce mode peut exposer des fonctionnalités, des données ou des points d'entrée (endpoints) de test qui ne sont pas destinés aux utilisateurs finaux, créant des failles de sécurité inattendues. (Note : L'APK ne peut être installé qu'avec la commande adb install -t).
- **Remédiation :** Désactiver l'attribut android:testOnly pour générer un APK de production valide.

3. **Signature avec un Certificat de Débogage**

- **Sévérité : Élevée**
- **Catégorie MASVS :** MSTG-CODE-1 *(L'application doit être signée avec un certificat valide)*
- **Description :** L'application est signée avec le certificat de développement par défaut d'Android (Android Debug).
- `Preuve : Rapport des certificats : Issuer: CN=Android Debug, O=Android, C=US`
- **Impact :** Les certificats de débogage sont connus de tous. Une application en production ne doit jamais utiliser ce certificat, car il ne garantit pas l'intégrité ni l'authenticité de l'auteur de l'application.
- **Remédiation :** Signer l'application avec un certificat Release unique, généré via un Keystore sécurisé appartenant à l'organisation.

4. **Sauvegarde des données de l'application autorisée**

- **Sévérité : Moyenne**
- **Catégorie MASVS :** MSTG-STORAGE-2
- **Description :** L'application autorise le système d'exploitation à sauvegarder ses données privées.
- `Preuve : Fichier AndroidManifest.xml, attribut android:allowBackup="true"`
- **Impact :** Un utilisateur malveillant ayant un accès physique à l'appareil déverrouillé peut utiliser l'outil ADB (Android Debug Bridge) pour extraire toutes les données privées de l'application (bases de données locales, préférences, jetons de session) sur un ordinateur, même sans accès root.
- **Remédiation :** Définir android:allowBackup="false". Si la sauvegarde est strictement nécessaire pour le fonctionnement, implémenter des règles d'exclusion (android:fullBackupContent) pour ignorer les fichiers contenant des données sensibles.

Autres observations

- **Composant exporté mal protégé :** Le ProfileInstallReceiver est exporté et protégé par la permission android.permission.DUMP. S'agissant d'une permission système (niveau signature/privilégié), le risque est modéré, mais il est recommandé de s'assurer que ce composant (souvent généré automatiquement par certaines bibliothèques) n'expose pas de fonctionnalités critiques.
- **Échec partiel de l'analyse (Scan SAST) :** Les logs d'analyse (page 11) indiquent une erreur du moteur d'analyse de code source : libsast scan failed.
- **Mécanismes anti-analyse détectés :** Des vérifications de l'environnement (Anti-VM) ont été détectées dans le code (vérification du Build.FINGERPRINT, MODEL, MANUFACTURER), ce qui indique une volonté de l'application d'empêcher son exécution dans des émulateurs.

Recommandations prioritaires

1. **Revoir le processus de Build (Release vs Debug) :** Les vulnérabilités principales (Debuggable, TestOnly, Debug Certificate) proviennent toutes du fait qu'il s'agit d'un build de développement. Générer une version "Release" signée correctement corrigera automatiquement ces trois vulnérabilités majeures.
2. **Désactiver la sauvegarde ADB :** Passer l'attribut allowBackup à false dans le Manifest pour protéger les données locales des utilisateurs contre l'extraction physique.

3. **Investigation complémentaire du code :** Étant donné l'échec du module SAST de MobSF, effectuer une revue manuelle ou via un autre outil statique du code décompilé (classes.dex) pour garantir l'absence de secrets "hardcodés".

Annexes

- **Liste des permissions demandées :**
  - `com.example.mypizzaworld.DYNAMIC_RECEIVER_NOT_EXPORTED_PERMISSION` *(Permission générée automatiquement pour gérer la sécurité des receivers dynamiques dans les versions récentes d'Android).*
  - android.permission.DUMP *(Vérifiée par un Receiver, niveau de protection système).*